

Document number: P3306R0.

Date: 2024-5-21.

Authors: Gonzalo Brito Gadeschi, Damien Lebrun-Grandie.

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>.

Audience: SG1, SG6.

Table of Contents

- Atomic Read-Modify-Write Improvements
- Motivation
- Design
- Wording

Atomic Read-Modify-Write Improvements

Extends read-modify-write atomic APIs with commonly requested functionality that is already available in some pre-existing implementations as an extension: integer shl, shr, mod, nand, div, mul, and floating-point div and mul.

Motivation

Without these extensions, users are required to implement this functionality by themselves on top of the atomic types, e.g., via user-specified CAS-loops. These loops complicate compiler optimizations and high-quality forward progress.

Design

The following APIs are commonly requested, and available in some implementations, like Kokkos `atomic_fetch_<op>` (https://kokkos.org/kokkos-core-wiki/API/core/atomics/atomic_fetch_op.html) or Rust's Atomic (<https://doc.rust-lang.org/std/sync/atomic/struct.AtomicU8.html>):

- Integer shl, shr, mod, nand.
- Integer and floating-point div, mul.

These follow the same semantics as the scalar operations, e.g., in terms of what uses exhibit undefined or implementation-defined behavior.

Wording

Add feature test macro to `<version>` synopsis:

```
#define __cpp_lib_atomic_fetch_ext 202XXXL // also in <atomic>
```

Add following functions for `<op>` in `{mod, nand, div, mul}` , immediately below `atomic_fetch_xor_explicit` :

```
namespace std {
// 33.5.9, non-member functions
...
template<class T>
T atomic_fetch_<op>(volatile atomic<T>*, // freestanding
                    typename atomic<T>::value_type) noexcept;
template<class T>
T atomic_fetch_<op>(atomic<T>*, // freestanding
                    typename atomic<T>::value_type) noexcept;
template<class T>
T atomic_fetch_<op>_explicit(volatile atomic<T>*, // freestanding
                             typename atomic<T>::value_type,
                             memory_order) noexcept;
template<class T>
T atomic_fetch_<op>_explicit(atomic<T>*, // freestanding
                             typename atomic<T>::value_type,
                             memory_order) noexcept;
...
}
```

Add following functions for `<op>` in `{shl, shr}` , immediately afterwards:

```

namespace std {
// 33.5.9, non-member functions
...
template<class T>
T atomic_fetch_<op>(volatile atomic<T>*, // freestanding
                    const unsigned) noexcept;
template<class T>
T atomic_fetch_<op>(atomic<T>*, // freestanding
                    const unsigned) noexcept;
template<class T>
T atomic_fetch_<op>_explicit(volatile atomic<T>*, // freestanding
                             const unsigned,
                             memory_order) noexcept;
template<class T>
T atomic_fetch_<op>_explicit(atomic<T>*, // freestanding
                             const unsigned,
                             memory_order) noexcept;
...
}

```

Add following functions immediately afterwards:

```

...
template<class T, typename BinaryOp>
T atomic_fetch_<op>(volatile atomic<T>*, BinaryOp&& bop, // freestanding
                  typename atomic<T>::value_type) noexcept;
template<class T, typename BinaryOp>
T atomic_fetch_<op>(atomic<T>*, // freestanding
                  typename atomic<T>::value_type) noexcept;
template<class T, typename BinaryOp>
T atomic_fetch_<op>_explicit(volatile atomic<T>*, // freestanding
                             typename atomic<T>::value_type,
                             memory_order) noexcept;
template<class T, typename BinaryOp>
T atomic_fetch_<op>_explicit(atomic<T>*, // freestanding
                             typename atomic<T>::value_type,
                             memory_order) noexcept;
...

```

Modify `tab:atomic.types.int.comp` (<https://eel.is/c++draft/atomics#tab:atomic.types.int.comp>) as follows:

key	op	computation
add	+	addition
sub	-	subtraction
<u>mul</u>	*	<u>multiplication</u>
<u>div</u>	/	<u>division</u>
<u>mod</u>	%	<u>modulo</u>
max		maximum
min		minimum
and	&	bitwise and
or		bitwise inclusive or
xor	^	bitwise exclusive or
<u>nand</u>		<u>bitwise and-not</u>
<u>shl</u>	<<	<u>left shift</u>
<u>shr</u>	>>	<u>right shift</u>

For each <op> in {mod, nand, div, mul} , add following public functions to [atomics.ref.int] specialization, immediately below fetch_xor :

```
namespace std {
template <> struct atomic_ref<integral> {
...
integral fetch_<op>(integral, memory_order = memory_order::seq_cst) const no
integral fetch_<op>(integral, memory_order = memory_order::seq_cst) const no
...
};
}
```

For each <op> in {shl, shr} , add following public functions to [atomics.ref.int] specialization, immediately below fetch_xor :

```

namespace std {
template <> struct atomic_ref<integral> {
...
integral fetch_<op>(const unsigned, memory_order = memory_order::seq_cst) co
integral fetch_<op>(const unsigned, memory_order = memory_order::seq_cst) co
...
};
}

```

For each <op> in {mul, div} , add following public functions to [atomics.ref.float] specialization, immediately below fetch_sub :

```

namespace std {
template <> struct atomic_ref<floating-point> {
...
floating-point fetch_<op>(floating-point, memory_order = memory_order::seq_c
floating-point fetch_<op>(floating-point, memory_order = memory_order::seq_c
...
};
}

```

For each <op> in {mod, nand, div, mul} , add following public functions to [atomics.types.int] specialization, immediately below fetch_xor :

```

namespace std {
template <> struct atomic<integral> {
...
integral fetch_<op>(integral, memory_order = memory_order::seq_cst) const no
integral fetch_<op>(integral, memory_order = memory_order::seq_cst) const no
...
};
}

```

For each <op> in {shl, shr} , add following public functions to [atomics.types.int] specialization, immediately below fetch_xor :

```

namespace std {
template <> struct atomic<integral> {
...
integral fetch_<op>(const unsigned, memory_order = memory_order::seq_cst) co
integral fetch_<op>(const unsigned, memory_order = memory_order::seq_cst) co
...
};
}

```

For each <op> in {mul, div} , add following public functions to [atomics.types.float] specialization, immediately below fetch_sub :

```

namespace std {
template <> struct atomic<floating-point> {
...
floating-point fetch_<op>(floating-point, memory_order = memory_order::seq_c
floating-point fetch_<op>(floating-point, memory_order success = memory_orde
...
};
}

```